

Умный роутинг выплат

Команда «Коломот» · Фёдор Филитович и Артём Корсаков

Презентация — ровно 9 слайдов, столько же ушло организаторам. Всё, что может понадобиться на вопросы, лежит здесь, в говоруне, а не в слайдах. Речь рассчитана на 9 минут 25 секунд — остаётся полминуты запаса.

Тайминг и роли

Время	Слайд	Кто	Тезис, ради которого слайд существует
0:00–0:15	1. Титул	Артём	Кто мы и что покажем
0:15–1:10	2. Задача	Артём	Три разных вопроса и конфликт целей из ТЗ
1:10–2:20	3. Архитектура	Фёдор	Два слоя, стратегия не может нарушить лимит
2:20–3:40	4. Правила	Фёдор	9 правил допуска и 7 целей — все из ТЗ
3:40–5:00	5. Согласование	Фёдор	Скоринг и три явных правила разрешения конфликта
5:00–6:20	6. Объяснимость	Фёдор	Разбор балла по слагаемым, каскад и fallback
6:20–7:15	7. Лимиты	Фёдор	Резервируем — наша проверка строже валидатора
7:15–8:35	8. Аналитика	Артём	Дрейф конверсии, рекомендации, backtest
8:35–9:25	9. Готовность	Артём	0 строк кода, 122 теста, готовы к вопросам

Артём открывает и закрывает, ведёт постановку задачи и аналитику. Фёдор — всё техническое ядро: архитектура, правила, скоринг, объяснимость, лимиты. Вопросы по коду всегда Фёдору. Передача слова — на смене слайда, вслух не объявляем.

Если начали отставать

Режем слайд 7 «Лимиты» до двух фраз (наивно продаёт лимит десять раз — мы резервируем; валидатор 29 из 29) и убираем из слайда 4 разбор четвертой и шестой стратегий. Это экономит около минуты и не ломает логику рассказа.

Что говорить

1 Титул

АРТЁМ

0:00–0:15 · 15 с

Команда «Коломот», задача «Умный роутинг выплат». Решение на Ruby, без единой внешней зависимости. За десять минут покажем, как оно выбирает провайдера, что делает при отказе и почему каждое решение можно объяснить числами.

Не зачитывать титул вслух

Слайд читается сам. Сразу переходим ко второму.

2

«Выбрать подходящего партнёра» — это три разных вопроса

АРТЁМ

0:15–1:10 · 55 с

«Выбрать наиболее подходящего партнёра» — это на самом деле три разных вопроса. Кому вообще **можно** отправить — ответ бинарный, нарушать нельзя никогда. Кого из них **лучше** выбрать — ответ всегда компромисс. Что делать, если провайдер **отказал** — нужна последовательность попыток.

Большинство ошибок в таких системах от того, что все три решают одним механизмом.

Конфликт целей здесь не побочный эффект, а суть задачи. Пример прямо из вашего ТЗ: цель — сорок процентов на vírau, а по факту vírau почти исчерпал дневной максимум, и одновременно на raufLOW висит обязательство два миллиона рублей в сутки. Обе цели корректны, одновременно выполнить нельзя. Дальше — про то, как мы это разгружаем.

3

Два слоя, которые нельзя смешивать

ФЁДОР

1:10–2:20 · 70 с

Поэтому в коде два независимых слоя. Hard-constraints — это допуск, они применяются **до** выбора стратегии. Soft-goals — это предпочтение, они работают только внутри уже допустимого пула.

Путь одной заявки. Сначала двигаем модельные часы и освобождаем слоты завершившихся заявок. Потом девять жёстких правил, и каждый отсев сразу пишется в attempts. Потом семь стратегий, каждая даёт свой под-скор от нуля до единицы. Композитный скоринг сводит их в одно число. Дальше каскад: отказ — следующий кандидат, никто не принял — self-provider. В конце занимаем слот и резервируем лимит.

Главное здесь вот что: стратегия физически не может уговорить роутер нарушить лимит. Альтернативу — единый скоринг, где нарушение просто дорого стоит — мы отвергли осознанно: там нарушение перестаёт быть невозможным.

4

Девять правил допуска и семь целей — все из ТЗ

ФЁДОР

2:20–3:40 · 80 с

Слева — девять жёстких ограничений. Все проверки из ТЗ плюс одна наша: потолок оборота. Финансовое обязательство «не более X в сутки» — это не то же самое, что технический дневной лимит, смешивать их нельзя. Каждое правило — отдельный класс со своим кодом причины, и отсев всегда объяснён числами: «сто пятьдесят тысяч больше лимита в пятьдесят тысяч».

Справа — все семь стратегий из ТЗ. Обращу внимание на две, где легко срезать угол.

Четвёртая, по сумме чека: у нас диапазон влияет на **предпочтение**, а не только отсекает. Провайдер вне своего диапазона остаётся допустимым, просто получает меньший балл.

Шестая, по интенсивности: текущая загрузка влияет на **выбор**, а не только проверяется на достижение потолка. Жёсткая часть живёт отдельно, в слое допуска.

И правила, и веса целей настраиваются конфигом, без правки кода.

Это два подкритерия жюри по три балла каждый

«Диапазон влияет на выбор, а не только используется как ограничение» и «загрузка влияет на выбор, а не только проверяется достижение лимита». Проговорить обязательно.

5

Учитываем всё сразу — и разрешаем конфликт явно

ФЁДОР

3:40–5:00 · 80 с

Как учитываем всё сразу: взвешенная сумма нормированных под-скоров.

Веса профиля `balanced` — вот они. Доли по количеству и объёму вместе дают сорок два процента, конверсия — восемнадцать. Веса живут в конфиге, а не в коде: завтра приоритетом станет конверсия — меняется одна строка, профиль `conversion_first` уже лежит в репозитории.

ТЗ требует определить поведение в трёх ситуациях, и все три закрыты явно.

Первая: цели указывают на разных провайдеров, а разрыв баллов мал.

Работает явный порядок политик. Финансовые обязательства стоят выше

целевых долей — за недобор оборота платят деньгами по договору, а долю можно добрать следующими заявками.

Вторая: цели равны полностью. Детерминированный разрыв — сначала priority, потом имя. Один и тот же вход всегда даёт один и тот же выход, это отдельный тест.

Третья: цель невыполнима, потому что нужный провайдер отсечён жёсткими ограничениями. Тогда цель не блокирует роутинг: заявка уходит дальше, а расходование попадает в отчёт с конкретной рекомендацией.

6

Почему выбран он, почему не остальные и что при отказе

ФЁДОР

5:00–6:20 · 80 с

Живой пример. Заявка на пятнадцать тысяч, Сбербанк, допустимы все трое — и цели расходятся: по диапазону суммы лучше rayflow, по загрузке quickray, по долям и конверсии viraу.

Скоринг сводит это в одно число: ноль семьдесят семь против ноль сорок семь и ноль пятнадцать. И в файле решения лежит не просто «выбрали viraу», а разложение балла по слагаемым — видно, что именно решило выбор. У исключённых своя причина: превышен лимит суммы, банка нет в списке, проиграл по баллу — и расшифровка числами. Тест требует, чтобы каждый допустимый провайдер получил запись: не молчим ни про кого.

Теперь отказ. Мы различаем два события, которые в ТЗ названы одним словом. Провайдер не принял заявку — это ретраибельный отказ, идём к следующему, а если их не осталось — на self-provider. Провайдер принял и обработал — это терминальный исход, маршрут уже не меняется.

Вот каскад в действии: двое отсеяны по лимиту суммы, третий отказался, заявка ушла на spacerauments.

Если спросят, почему в сдаваемом прогоне ноль fallback

В сдаваемом профиле искусственные отказы выключены осознанно: эталонный файл требует для op_103 строго quickray, а он там единственный допустимый — случайный отказ увёл бы заявку в spacerauments и провалил проверку. Механика никуда не делась: команда **rake demo** и четыре теста. Предложить показать прямо сейчас.

7

Лимиты резервируем, а не считаем пост-фактум

ФЁДОР
6:20–7:15 · 55 с

Одна деталь, которая обычно ломает такие системы. Наивная реализация увеличивает оборот только после подтверждения выплаты. Тогда десять заявок, отправленных подряд, все проходят проверку дневного лимита — и лимит оказывается продан десять раз. Мы резервируем сумму в момент выбора провайдера и снимаем резерв при завершении. То же самое для одновременных заявок.

Побочная выгода, важная для проверки: ваш валидатор считает лимиты по исходному снимку. Наша проверка строго жёстче — значит, выбранный нами провайдер гарантированно входит в его список допустимых. На публичной очереди: двадцать девять проверок, ноль ошибок, ноль предупреждений.

8

Отчёт называет причину расхождения и параметр к изменению

АРТЁМ
7:15–8:35 · 80 с

Аналитика. Самое интересное — вот это. Заявленная конверсия payflow — ноль девяносто один, самая высокая из трёх. Фактическая по ста историческим операциям — ноль сорок семь. Дрейф минус сорок три пункта. Провайдер с лучшей заявленной конверсией на деле худший.

Роутинг, который берёт conversion_24h как есть, будет систематически слать заявки туда, где они не проходят. Поэтому у нас конверсия смешанная, а дрейф вынесен в отчёт отдельным разделом.

И рекомендации не «стоит улучшить», а провайдер, число и параметр: «payflow выбрал девяносто восемь процентов дневного лимита — поднять daily_amount_limit до трёх миллионов шестисот тысяч или снизить долю трафика».

И мы не просто утверждаем, что роутинг хороший — мы переиграли через него сто исторических операций. Суммарное отклонение от целевых долей упало с тридцати двух пунктов до двадцати шести. И отдельная находка: двадцать девять из ста исторических выплат ушли провайдеру, который по правилам допуска вообще не мог их принять.

9

Что меняется без кода — и как всё проверить

АРТЁМ

8:35–9:25 · 50 с

Про гибкость коротко. Новый провайдер, другие лимиты, другие веса, другой порядок разрешения конфликтов, целиком другая политика — это ноль строк кода, только конфиг и данные. Новое правило или новая стратегия — класс и строка в реестре. Ни одно имя провайдера в коде не зашито.

Сто двадцать два теста без единой внешней зависимости, валидатор двадцать девять из двадцати девяти, CI зелёный на каждый push. Всё, что мы показали, проверяется командами на слайде.

Ограничения кейса соблюдены: нейросетей внутри решения нет, проприетарного нет, весь код на Ruby, повторный запуск на тех же данных даёт побайтово тот же результат.

Спасибо. Готовы к вопросам.

Ответы на ожидаемые вопросы

Отвечает тот, чья зона. Всё по коду — Фёдор.

— Почему именно такие веса? Не подобрали ли вы их под данные?

Веса — расстановка приоритетов, а не магия: доли по количеству и объёму вместе 0.42, это прямое требование договора; конверсия 0.18. Главный аргумент: на истории профили traffic, volume, commitments и balanced дают одинаковые 26 п.п. — результат устойчив к перекосу весов, а не держится на одной подобранной комбинации. **Разбор каждого веса — в следующем разделе.**

— Что будет, если завтра добавится провайдер?

Строка в providers.json, кода не трогаем. Стратегия долей сама пересчитает целевое распределение, каскад пересчитает ранги среди доступных.

— У вас в сдаваемом прогоне ноль fallback. Каскад вообще работает?

Работает: **rake demo** — восемь повторных попыток, четыре ухода на self-provider, плюс четыре теста в router_test.rb. В сдаваемом профиле искусственные отказы исключены, потому что эталонный файл требует для op_103 строго quickrau. Предложить показать.

— Вы используете нейросети?

Внутри решения — нет, и запрет в ТЗ относится именно к решению. Весь выбор провайдера — детерминированные правила и арифметика, ни одной модели и ни одного внешнего сервиса решение не вызывает: у него вообще нет внешних зависимостей.

Повторный запуск даёт побайтово тот же результат, это закреплено тестом test_run_is_reproducible.

— Почему quickrau отклонился от целевой доли на 5 пунктов?

Это минимально достижимое отклонение. Четыре заявки из десяти безальтернативны по жёстким ограничениям, а при цели 25 % и десяти заявках доля кратна 10 п.п., так что ближе подойти арифметически нельзя. Отчёт говорит это сам, в поле deviations_note.

— Ваш валидатор и скрытая проверка считают лимиты по-разному — не ломает ли это вас?

Мы намеренно строже: резервируем лимит в момент выбора, а проверка считает по исходному снимку. Строже — значит наш выбор всегда входит в её список допустимых. Обратное было бы опасно.

— А если тестовая очередь будет длинной?

У нас модельное время: заявка держит слот latency_sec и освобождает его. На длинной очереди это решает, упрётся провайдер в лимит одновременных заявок или нет. Плюс резервирование не даёт продать дневной лимит дважды.

— Что будет с битыми входными данными?

Битый JSON, amount строкой или отрицательным, дубли operation_id, providers.json без обязательного поля, неизвестная стратегия в конфиге — всё это внятная ошибка с указанием причины, а не стектрейс, и всё закрыто тестами. **Полный список — в следующем разделе.**

— Что если неуспешная выплата должна вести к перемаршрутизации?

Мы выбрали более строгую трактовку — «принял и обработал» это финал.

Противоположное прочтение реализовано заранее: флаг `retry_on_terminal_failure` в конфиге, по умолчанию выключен, покрыт четырьмя тестами. Файлы регенерируются одной командой, кода менять не нужно.

— Организаторы отвечали вам на эти вопросы?

Отдельной QA-сессии с ответами не было. По каждому спорному месту мы зафиксировали трактовку сами, выбрали более строгий вариант и записали обоснование в `docs/QA-ANSWERS.md` — там же указано, что переключается, если прочтение окажется другим.

— Почему не сделали ML-модель выбора?

Условия кейса это прямо запрещают. И по существу: выплаты требуют аудируемости — на каждое решение нужно уметь показать, какое правило и какое число его определили.

Развёрнутые ответы

Это было четырем запасными слайдами. В презентацию, которая ушла организаторам, они не попали — держим здесь и отвечаем словами.

Почему веса именно такие

- Веса — не подобранная магия, а расстановка приоритетов, которую можно защитить словами. **Доли по количеству и объёму вместе дают 0.42** — это прямое требование ТЗ и договора.
- **Конверсия 0.18** — второй по величине одиночный вес: отправлять туда, где выплата не проходит, дороже, чем немного отклониться от доли.
- **Фин. обязательства 0.14** — вес умеренный, потому что стратегия и так разрывная: она выстреливает только когда минимум ещё не набран.
- **Каскад, диапазон суммы, загрузка — 0.26 на троих**: это уточняющие сигналы, жёсткие версии этих же ограничений уже отработали в слое допуска.
- Главный аргумент против «подогнали под данные»: на истории профили **traffic**, **volume**, **commitments** и **balanced** дают одинаковые 26 п.п. Результат устойчив к перекосу весов.

Нормировка: сравнительные и абсолютные сигналы

Сравнительные — min-max по пулу: traffic_share, volume_share, conversion, cascade_priority. Разброс конверсий в кейсе — 0.79...0.91. Без нормировки эта разница растворяется среди остальных слагаемых и не влияет ни на что.

Абсолютные — по своей шкале: load_balance, turnover_commitment, amount_band. «Загружен на 64 %» осмысленно само по себе, сравнивать не с чем и не нужно.

Сказать честно, если спросят про два кандидата

При двух кандидатах сравнительные стратегии дают ровно 1.0 и 0.0, и выбор превращается во взвешенное голосование. На публичной очереди из 10 заявок это видно: 4 заявки безальтернативны, на остальных шести все содержательные профили сходятся к одному решению. Расхождение политик проявляется на объёме — поэтому у нас есть backtest.

Обработка ошибок и граничных ситуаций

Ситуация	Поведение
Файл не найден, битый JSON	Ошибка с путём и причиной разбора, а не стектрейс
amount строкой, нулём, отрицательным; заявка без id	Отклоняется на загрузке с указанием поля и значения
Дубли operation_id	Отклоняются: молча смаршрутизировать дубль хуже, чем упасть
Очередь — один объект вместо массива	Принимается: валидатор организаторов допускает оба варианта
Пустая очередь	Валидный отчёт с нулями, без деления на ноль
providers.json не массив / без обязательного поля	Ошибка с указанием, чего именно не хватает

Ситуация	Поведение
Неизвестная стратегия или правило в конфиге	Ошибка при старте, а не тихое игнорирование
Под заявку не подходит вообще никто	Уходит на spacpayments, помечается fallback_used
daily_amount_limit = null	Отсутствие лимита, а не ноль
У провайдера кончились реквизиты посреди очереди	Покидает пул, остальные заявки идут дальше

Всё перечисленное закрыто тестами: 11 в loaders_test.rb, 9 в config_test.rb, 18 в report_test.rb, 17 в router_test.rb.

Спорные места ТЗ: что выбрали и что переключается

Вопрос	Наша трактовка	Если прочтение другое
Неуспешная выплата — отказ, требующий перемаршрутизации, или финал?	Финал. К следующему переходим, только если провайдер не принял заявку в работу	Флаг retry_on_terminal_failure — реализован, покрыт тестами, по умолчанию выключен
Скрытая проверка считает лимиты по исходному снимку providers.json?	Да — поэтому мы намеренно строже: резервируем лимит в момент выбора	Ослабить резервирование в конфиге; менее строгий вариант нам не нужен
Запрет нейросетей — про решение или про написание кода?	Про решение: внутри проекта моделей нет, только правила и арифметика	Ничего: решение не вызывает ни одной модели и ни одного внешнего сервиса
Можно ли добавлять свои поля в решение?	Да, ТЗ прямо разрешает дополнительные поля	Убрать score, score_breakdown, stage, eligible_providers из Decision#to_h

Отдельной QA-сессии с ответами организаторов не было. Обоснование по каждому пункту — в docs/QA-ANSWERS.md.

Перед выходом

- [] Прогнать вслух дважды с таймером. Первый прогон почти наверняка будет длиннее — резать надо формулировки, а не слайды.
- [] kolomot-slides.pdf лежит локально на ноутбуке, а не только в облаке. Открыть в полноэкранном режиме заранее.
- [] Второе окно: терминал в каталоге проекта, **rake demo** и **rake validate** уже прогнаны один раз, чтобы не ждать при показе.
- [] Открытая вкладка с репозиторием на GitHub, ветка main.
- [] Договориться, кто держит время и подаёт знак на пятой минуте.

После защиты — стопкод

Регламент полностью — в TASKS-ARTEM.md, раздел 5. Коротко и вдвоём:

```
ср <выданный файл> data/operations_queue_test.json
rake submit
rake validate FILE=routing_decisions_test.json
```

Дальше: коммит и push в main, дождаться зелёного CI и **глазами** увидеть оба файла в корне main на GitHub. Неправильное имя, не тот каталог или не та ветка = ноль баллов из сорока. Проверяем оба, вдвоём, по списку.